OpenCL Student Handout

Core Concepts, Examples, and Lecture Notes

Prepared for PDC students

Contents

1 Global and Local Work-Group Sizes

1.1 The main idea

When we launch an OpenCL kernel, we define an **NDRange**, which is the index space over which work-items execute. In the 1D case, two quantities matter most:

- **Global work size**: the total number of work-items.
- **Local work size**: the number of work-items in each work-group.

A useful sentence to remember is:

> Global size tells us *how much total work exists*. Local size tells us *how that work is partitioned into groups*.

1.2 Basic relationship

If the global size is 1024 and the local size is 64, then the number of work-groups is

$$\text{number of work-groups} = \frac{1024}{64} = 16.$$

So OpenCL creates:

- 1024 total work-items,
- 16 work-groups,
- 64 work-items in each work-group.

1.3 Why local size matters

Students often ask: if the global size already tells us how many work-items there are, why do we need local size? The answer is that the work-group is the unit of:

- **synchronization** — only work-items in the same work-group can synchronize with `barrier()`,
- **local memory sharing** — work-items in the same group can cooperate through `__local` memory,
- **scheduling** — hardware and runtimes typically schedule work in groups, not as completely independent single work-items.

1.4 Example: vector addition

For

$$C[i] = A[i] + B[i],$$

a natural design is one work-item per output element.

```
__kernel void vadd(__global const float* A,
                   __global const float* B,
                   __global float* C,
                   const int N)
{
    size_t gid = get_global_id(0);
    if (gid < (size_t)N) {
        C[gid] = A[gid] + B[gid];
    }
}
```

On the host side, a common safe pattern is:

```
size_t localSize = 64;
size_t globalSize = ((N + localSize - 1) / localSize) * localSize;
```

This rounds the global size up to a multiple of the local size. If $N = 1000$ and localSize $= 64$, then

$$\text{globalSize} = \left\lceil \frac{1000}{64} \right\rceil \times 64 = 1024.$$

The extra work-items are harmless because the kernel checks `gid < N`.

1.5 How to choose global size

Global size is usually determined by the **problem size**:

- vector length $N \rightarrow$ global size $= N$,
- image of width $\times$ height $\rightarrow$ 2D global size $= \{\text{width}, \text{height}\}$,
- matrix output of size $M \times N \rightarrow$ 2D global size $= \{M, N\}$.

1.6 How to choose local size

Local size depends on hardware and kernel behavior. A good practical approach is:

1. Query device constraints such as max work-group size and preferred multiple.
2. Start with device-friendly candidates such as 32, 64, 128, or 256.
3. Measure performance.
4. Choose the best among correct configurations.

On the Intel GPU we inspected, the preferred multiple was 32 and max work-group size was 256, so sensible first choices are 32, 64, 128, and 256.

1.7 Likely questions

Q: Should local size always divide global size exactly?
If you specify an explicit local size, we usually round global size up to a multiple of local size so all work-groups are complete.

Q: If I choose larger local size, will performance always improve?
No. Very small groups may underutilize hardware, while very large groups may consume too many resources or reduce concurrency.

Q: Can I let the runtime choose local size?
Yes. Passing `nullptr` for local size is valid and often useful for first experiments, though it may not give the best performance.

2 `get_global_id`, `get_local_id`, `get_group_id`, and NDRange Indexing

2.1 The three main IDs

Each work-item needs to know where it is:

- in the whole problem space,
- inside its own work-group,
- and which work-group it belongs to.

That is why OpenCL provides:

- `get_global_id(dim)`,
- `get_local_id(dim)`,
- `get_group_id(dim)`.

2.2 1D visual example

Suppose global size is 16 and local size is 4. Then OpenCL creates four work-groups:

Global ID	Group ID	Local ID
0	0	0
1	0	1
2	0	2
3	0	3
4	1	0
5	1	1
6	1	2
7	1	3
8	2	0
9	2	1
10	2	2
11	2	3
12	3	0
13	3	1
14	3	2
15	3	3

This is the most important pattern to understand.

2.3 Meaning of each function

`get_global_id(0)` gives the absolute position in the whole NDRange.
`get_local_id(0)` gives the position inside the current work-group.
`get_group_id(0)` gives the work-group number.

2.4 Relationship between them

For a 1D NDRange,

$$\text{global\_id} = \text{group\_id} \times \text{local\_size} + \text{local\_id}.$$

This is exactly analogous to CUDA's

```
gid = blockIdx.x * blockDim.x + threadIdx.x;
```

In OpenCL, we usually do not compute it manually because `get_global_id(0)` already provides it.

2.5 Example: vector addition indexing

```
__kernel void vadd(__global const float* A,
                   __global const float* B,
                   __global float* C,
                   const int N)
{
    size_t gid = get_global_id(0);
    if (gid < (size_t)N) {
        C[gid] = A[gid] + B[gid];
    }
}
```

The key idea is that work-item `gid` computes output element `C[gid]`.

2.6 2D indexing

For matrices and images, 2D NDRange is more natural:

```
int x = get_global_id(0);
int y = get_global_id(1);
```

For image processing, `x` may represent the column and `y` the row.

```
__kernel void brighten(__global const float* in,
                       __global float* out,
                       const int width,
                       const int height)
{
    int x = get_global_id(0);
    int y = get_global_id(1);

    if (x < width && y < height) {
        int idx = y * width + x;
        out[idx] = in[idx] * 1.2f;
    }
}
```

2.7 Likely questions

Q: Why do we need both global ID and local ID?
Because they answer different questions. Global ID tells a work-item where it is in the full problem; local ID tells it where it is inside its own group.

Q: Can two work-items have the same local ID?
Yes, if they belong to different groups. Local ID is unique only within a work-group.

Q: Can two work-items have the same global ID?
No. Global ID is unique in the whole NDRange.

Q: What is the most common beginner mistake here?
Using local ID where global ID is required for full-domain indexing.

3 Private vs Local vs Global Variables

3.1 Main distinction

The easiest way to remember the OpenCL memory scopes is:

- **Private** — belongs to one work-item only.
- **Local** — shared by all work-items in one work-group.
- **Global** — visible in device-wide memory across the kernel launch.

The key question is always:

> Who can access this variable?

3.2 Private variables

A variable declared inside a kernel such as

```
float sum = 0.0f;
```

is usually **private**. Every work-item gets its own copy.

Example:

```
__kernel void example(__global const float* A,
                      __global float* B)
{
    size_t gid = get_global_id(0);
    float temp = A[gid] * 2.0f;   // private
    B[gid] = temp;
}
```

Each work-item has its own `temp`. No other work-item can see it.

3.3 Local memory

OpenCL local memory is explicitly marked with `__local`. It is shared by work-items in the same work-group.

```
__kernel void local_copy(__global const float* A,
                         __global float* B,
                         __local float* temp)
{
    size_t gid = get_global_id(0);
    size_t lid = get_local_id(0);

    temp[lid] = A[gid];
    barrier(CLK_LOCAL_MEM_FENCE);
    B[gid] = temp[lid];
}
```

Here `temp` is shared inside the work-group. That is why it is indexed by `lid`, not by the global ID.

3.4 Global memory

Kernel arguments such as `__global float* A` live in global memory. This is where large input and output arrays normally live.

```
__kernel void vadd(__global const float* A,
                   __global const float* B,
                   __global float* C,
                   const int N)
{
    size_t gid = get_global_id(0);
    if (gid < (size_t)N) {
        C[gid] = A[gid] + B[gid];
    }
}
```

3.5 A three-scope example

```
__kernel void demo(__global const float* A,
                   __global float* B,
                   __local float* tile)
{
    size_t gid = get_global_id(0);   // private
    size_t lid = get_local_id(0);    // private

    float temp = A[gid] * 2.0f;      // private
```

```
    tile[lid] = temp;                // local
    barrier(CLK_LOCAL_MEM_FENCE);

    B[gid] = tile[lid];              // global output
}
```

3.6 Likely questions

Q: Is a variable declared inside a kernel private or local?
Usually private, unless it is explicitly declared with `__local`.

Q: Can one work-item read another work-item's private variable?
No.

Q: Can one work-group access another group's local memory?
No. Local memory is private to the work-group.

Q: Why is local memory indexed with local ID?
Because local memory belongs to one work-group, not the whole NDRange.

4 Local Memory, `barrier()`, and Tiled Computation

4.1 Why local memory exists

If many work-items in the same work-group need overlapping data, reading that data repeatedly from global memory is wasteful. The idea is therefore:

> Load shared data once into local memory, then let the work-group reuse it.

This is especially useful in matrix multiplication, convolution, and stencil-style computations.

4.2 Why `barrier()` is needed

When work-items share local memory, one work-item may need data written by another. Without synchronization, a work-item may read before the write is complete. The barrier solves that problem.

```
__kernel void neighbor_read(__global const float* A,
                            __global float* B,
                            __local float* temp)
{
    size_t gid = get_global_id(0);
    size_t lid = get_local_id(0);
    size_t lsz = get_local_size(0);

    temp[lid] = A[gid];
    barrier(CLK_LOCAL_MEM_FENCE);

    B[gid] = temp[(lid + 1) % lsz];
}
```

Here the barrier ensures the whole work-group has finished filling `temp` before any work-item reads a neighbor's entry.

4.3 Important barrier rule

Every work-item in the work-group must encounter the barrier consistently. This is invalid:

```
if (get_local_id(0) < 8) {
    barrier(CLK_LOCAL_MEM_FENCE);
}
```

Only part of the group reaches the barrier, so behavior becomes invalid.

4.4 Tiled matrix multiplication

The classic use of local memory is tiled matrix multiplication. One work-group computes one tile of the output matrix, and the work-group cooperatively loads one tile of A and one tile of B into local memory.

```
#define TS 16

__kernel void matmul_tiled(__global const float* A,
                           __global const float* B,
                           __global float* C,
                           const int M,
                           const int N,
                           const int K)
{
    int row = get_global_id(0);
    int col = get_global_id(1);
    int lr  = get_local_id(0);
    int lc  = get_local_id(1);

    __local float tileA[TS][TS];
    __local float tileB[TS][TS];

    float sum = 0.0f;

    for (int t = 0; t < (K + TS - 1) / TS; t++) {
        int aCol = t * TS + lc;
        int bRow = t * TS + lr;

        if (row < M && aCol < K)
            tileA[lr][lc] = A[row * K + aCol];
        else
            tileA[lr][lc] = 0.0f;

        if (bRow < K && col < N)
            tileB[lr][lc] = B[bRow * N + col];
        else
            tileB[lr][lc] = 0.0f;

        barrier(CLK_LOCAL_MEM_FENCE);

        for (int k = 0; k < TS; k++) {
            sum += tileA[lr][k] * tileB[k][lc];
        }

        barrier(CLK_LOCAL_MEM_FENCE);
    }

    if (row < M && col < N)
        C[row * N + col] = sum;
}
```

4.5 How to explain the two barriers

There are two barriers inside the tile loop because:

- the first barrier ensures the tile is fully loaded before computation starts,
- the second barrier ensures everyone is finished using the tile before local memory is overwritten in the next loop iteration.

4.6 Likely questions

Q: Why not always use local memory?
Because local memory is limited, requires extra loads and synchronization, and only helps when data is reused.

Q: Why does vector addition usually not benefit much from local memory?
Because each element is typically used once, so there is little shared reuse to exploit.

Q: Is local memory always physically on-chip?
The safe explanation is to treat it as a work-group-shared abstraction. On many GPU implementations it maps to fast shared memory-like hardware, but the programming model meaning is more important than the exact physical promise.

5 Kernel Design

5.1 What kernel design really means

Kernel design means deciding:

- what one work-item should compute,
- how work-items map to problem indices,
- which data belongs in private, local, and global memory,
- whether work-items need cooperation,
- and what work-group shape makes sense.

A very useful sentence is:

> Kernel design is the mapping of an algorithm onto the OpenCL execution model.

5.2 First design question: what does one work-item do?

Before writing any kernel, ask: what is the unit of work?

- In vector addition, one work-item computes one output element.
- In matrix multiplication, one work-item may compute one output cell.
- In image processing, one work-item may compute one pixel.

That choice determines indexing, memory use, and whether cooperation is needed.

5.3 Example: a good simple design

Vector addition is a good first example because the mapping is direct:

```
__kernel void vadd(__global const float* A,
                   __global const float* B,
                   __global float* C,
                   const int N)
{
    size_t gid = get_global_id(0);
    if (gid < (size_t)N) {
        C[gid] = A[gid] + B[gid];
    }
}
```

This design is clean because it uses one work-item per output and requires no group cooperation.

5.4 Example: a bad design

```
__kernel void bad_vadd(__global const float* A,
                       __global const float* B,
                       __global float* C,
                       const int N)
{
    int gid = get_global_id(0);
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }
}
```

This is a poor design because every work-item redundantly repeats the whole loop.

5.5 Naive vs tiled matrix multiplication as design evolution

A good teaching pattern is:

1. Start with naive matrix multiplication: one work-item computes one output cell.
2. Observe repeated global-memory reuse.
3. Improve the design with work-group tiling and local memory.

This shows that optimization should follow correctness and clarity.

5.6 Likely questions

Q: Should one work-item always compute one output element?
Not always, but it is a very common and useful starting point.

Q: Is the most optimized kernel always the best first kernel to write?
No. First write a simple correct kernel, then optimize it.

Q: How do I know whether to use 1D or 2D NDRange?
Choose the dimensionality that matches the natural shape of the problem: vectors are usually 1D, images and matrices are often 2D.

6 Device- and Architecture-Specific Optimization

6.1 Portable code, non-portable performance

OpenCL is portable across CPUs, GPUs, and other accelerators, but those devices are architecturally different. So while the same kernel may run correctly on multiple devices, its performance can vary widely.

A key sentence to remember is:

> Portable code does not imply portable performance.

6.2 What differs across devices?

Different devices vary in:

- compute units,
- memory hierarchy,
- preferred work-group multiples,
- local memory size,
- vector capabilities,
- sensitivity to branching and synchronization.

That is why optimization must be informed by device queries.

6.3 Example from our machine

From the device queries performed in class:

- Intel GPU: 24 compute units, max work-group size 256, preferred multiple 32, local memory 64 KB.
- POCL CPU device: 4 compute units, a much larger max work-group size, but very different execution behavior.

This immediately suggests that work-group sizes that are good for the GPU may not be ideal for the CPU.

6.4 Common device-specific optimizations

- tune local work-group size,
- choose tile size based on local memory and subgroup preferences,
- use local memory when the device and algorithm justify it,
- adapt launch parameters based on device type.

6.5 Likely questions

Q: If OpenCL is portable, why do we still tune per device?
Because code portability means it can run, not that it will run optimally everywhere.

Q: Should CPU and GPU always use the same local size?
No. A value may be valid on both but not optimal on both.

Q: Is device-specific tuning against the spirit of OpenCL?
No. It is a normal part of performance engineering.

7 Debugging in OpenCL

7.1 A layered debugging mindset

The best debugging mindset is to work in layers:

1. platform and device discovery,
2. context and queue creation,
3. program build,
4. buffer creation and data transfer,
5. kernel argument setup,
6. kernel launch,
7. result read-back,
8. correctness check,
9. performance only after correctness.

This prevents confusion and false conclusions.

7.2 Always check error codes

Every important OpenCL call should be checked:

```
static void checkErr(cl_int err, const char *where) {
    if (err != CL_SUCCESS) {
        std::cerr << where << " failed with error " << err << "\n";
        std::exit(1);
    }
}
```

A good teaching line is:

> Unchecked error codes create fake mysteries.

7.3 Always print the selected platform and device

This matters because students often think they are running on one device while the program is actually running on another.

7.4 Print the first mismatch

When checking correctness, do not only print PASS or FAIL. Print the first mismatch with enough detail to interpret the bug.

```
bool ok = true;
for (int i = 0; i < N; i++) {
    float ref = A[i] + B[i];
    float diff = std::fabs(C[i] - ref);
    if (diff > 1e-5f) {
        std::cout << "Mismatch at i=" << i
                  << " A=" << A[i]
                  << " B=" << B[i]
                  << " C=" << C[i]
```

```
                  << " REF=" << ref
                  << " DIFF=" << diff << "\n";
        ok = false;
        break;
    }
}
```

7.5 Use tiny test sizes first

Before timing large sizes, use small values such as $N = 8$, $N = 10$, or $N = 16$. Small sizes make indexing mistakes and output patterns much easier to see.

7.6 A powerful debugging trick

Temporarily replace the real computation with something trivial such as:

```
C[gid] = (float)gid;
```

This isolates indexing and write-back logic from arithmetic logic.

7.7 Use CPU OpenCL as a baseline

If both GPU and CPU OpenCL are available, try the same kernel on CPU OpenCL. In our experiments, the POCL CPU path provided a correct baseline while the Intel GPU runtime produced incorrect results for this simple kernel. That showed the code logic was fine and the GPU runtime path was the real issue.

7.8 Likely questions

Q: What is the most common beginner bug?
Wrong indexing or wrong kernel argument setup.

Q: Why use tiny inputs first?
Because small inputs are easy to inspect and reason about.

Q: Can runtime or driver issues cause wrong output even if the kernel is correct?
Yes. Real-world heterogeneous computing includes runtime maturity and driver quality as practical factors.

8 Profiling and Performance Measurement

8.1 What we want to measure

In OpenCL, total runtime is usually not just kernel time. A full workflow may include:

- host-to-device transfer (H2D),
- kernel execution,
- device-to-host transfer (D2H),
- and total end-to-end time.

A key lesson is:

> A fast kernel does not automatically mean a fast application.

8.2 Enable queue profiling

To measure OpenCL command durations with events, create the queue with profiling enabled:

```
cl_command_queue queue = clCreateCommandQueue(
    context, device, CL_QUEUE_PROFILING_ENABLE, &err);
```

8.3 Event timing helper

```
static double event_ms(cl_event ev) {
    cl_ulong start = 0, end = 0;
    clGetEventProfilingInfo(ev, CL_PROFILING_COMMAND_START,
                            sizeof(start), &start, nullptr);
    clGetEventProfilingInfo(ev, CL_PROFILING_COMMAND_END,
                            sizeof(end), &end, nullptr);
    return (end - start) / 1e6;
}
```

8.4 Capturing write, kernel, and read times

```
cl_event evWriteA, evWriteB, evKernel, evRead;

clEnqueueWriteBuffer(queue, bufA, CL_TRUE, 0, bytes, A.data(),
                     0, nullptr, &evWriteA);
clEnqueueWriteBuffer(queue, bufB, CL_TRUE, 0, bytes, B.data(),
                     0, nullptr, &evWriteB);
clEnqueueNDRangeKernel(queue, kernel, 1, nullptr, &globalSize, &localSize,
                       0, nullptr, &evKernel);
clEnqueueReadBuffer(queue, bufC, CL_TRUE, 0, bytes, C.data(),
                    0, nullptr, &evRead);

double h2d_ms = event_ms(evWriteA) + event_ms(evWriteB);
double kernel_ms = event_ms(evKernel);
double d2h_ms = event_ms(evRead);
```

8.5 Why timing breakdown matters

Suppose the kernel time is very small, but transfers are large. Then the GPU may look excellent if we only report kernel time, but poor if we report end-to-end time. Both measurements are meaningful, but they answer different questions.

8.6 Warm-up and repeated trials

The first run may include extra overhead such as runtime initialization or JIT effects. So performance experiments should usually:

- do a warm-up run,
- run multiple measured trials,
- report average and perhaps minimum time.

8.7 Bandwidth and GFLOPS

For memory-bound kernels such as vector addition, effective bandwidth is often more informative than FLOP count. For compute-heavy kernels such as matrix multiplication, GFLOPS is more appropriate.

For vector addition with single-precision data, each element involves roughly 12 bytes of traffic (read A, read B, write C), so:

$$\text{Bandwidth} = \frac{12N}{\text{kernel time in seconds}}.$$

For matrix multiplication of size $M \times K$ by $K \times N$, a common FLOP estimate is:

$$2MKN,$$

so:

$$\text{GFLOPS} = \frac{2MKN}{\text{time in seconds} \times 10^9}.$$

8.8 Likely questions

Q: What is the most important time to report?
There is no single answer. Kernel time is useful for studying device execution, while end-to-end time is better for application usefulness.

Q: Why run multiple trials?
Because timings vary due to OS activity, runtime effects, and general noise.

Q: Why can GPU kernel time be small but total time large?
Because transfer and launch overhead may dominate.

9 Final Takeaways

The OpenCL programming model becomes easier to understand when each concept is tied to a clear question:

- Global vs local sizes: how much work exists, and how is it grouped?
- IDs and indexing: where is each work-item in the whole problem and in its group?
- Private, local, and global memory: who owns the data and who can access it?
- Local memory and barriers: when do work-items need to cooperate?
- Kernel design: what should one work-item compute, and how should work-groups be used?
- Device-specific optimization: what does this device prefer, and how should tuning change?
- Debugging: which layer is failing, and how do we isolate it?
- Profiling: where is time spent, and what is a fair comparison?

A strong mental model is more valuable than memorizing syntax. If the mapping between the problem, the work-items, the memory spaces, and the work-groups is clear, most OpenCL code becomes much easier to reason about.